

- Họ tên: Hồ Tuấn Huy
- MSSV: 20225856

Báo cáo kết quả bài tập Kiểm tra kiểu

I. Mô tả hàm compileConstant trong parser.c và ứng dụng của hàm đó trong kiểm tra kiểu

- Mô tả hàm compileConstant trong parser.c:

- Hàm **compileConstant()** được dùng để phân tích các hằng số trong chương trình KPL, đặc biệt là hằng có dấu. Hàm xác định hằng dựa trên **lookAhead->tokenType** và xử lý thông qua cấu trúc **switch**.
- Ví dụ, khi gặp dấu + thì hàm tiêu thụ token này và tiếp tục phân tích hằng phía sau bằng **compileConstant2()**:

```
case SB_PLUS:
    eat(SB_PLUS);
    constValue = compileConstant2();
    if (constValue->type == TP_INT) {
        // Valid integer constant
    } else {
        error(ERR_TYPE_INCONSISTENCY, currentToken->lineNo, currentToken->colNo);
    }
    break;
```

- Trong trường hợp dấu - thì sau khi phân tích hằng, nếu kiểu hợp lệ thì giá trị hằng được đổi dấu:

```
case SB_MINUS:
    eat(SB_MINUS);
    constValue = compileConstant2();
    if (constValue->type == TP_INT) {
        constValue->intValue = -constValue->intValue;
    } else {
        error(ERR_TYPE_INCONSISTENCY, currentToken->lineNo, currentToken->colNo);
    }
    break;
```

- Đối với hằng ký tự, hàm tạo trực tiếp giá trị hằng mà không thông qua **compileConstant2()**:

```
case TK_CHAR:
    eat(TK_CHAR);
    constValue = makeCharConstant(currentToken->string[0]);
    break;
```

- Các hằng không có dấu (số nguyên hoặc hằng đã khai báo) được xử lý trong nhánh mặc định thông qua **compileConstant2()**:

```
default:
    constValue = compileConstant2();
    break;
}
```

- Qua đó, **compileConstant()** đóng vai trò là hàm điều phối, xử lý dấu của hằng và chuyển phân phân tích chi tiết cho **compileConstant2()** khi cần thiết.
- **Ứng dụng của hàm đó trong kiểm tra kiểu:**
- **compileConstant()** đóng vai trò quan trọng trong việc kiểm tra kiểu dữ liệu đối với các hằng có dấu. Theo quy tắc của ngôn ngữ KPL, các toán tử đơn + và - chỉ được áp dụng cho hằng kiểu số nguyên. Do đó:
 - Các biểu thức như +500 hay -10000 là hợp lệ.
 - Các biểu thức như +'z' hay -c (c là hằng ký tự) là không hợp lệ => gây lỗi không tương thích kiểu.
 - Việc kiểm tra này được thực hiện ngay trong quá trình phân tích cú pháp, giúp phát hiện sớm lỗi ngữ nghĩa và đảm bảo tính nhất quán kiểu dữ liệu cho các bước xử lý tiếp theo.

II. Kết quả thực hiện với example12.kpl không có lỗi

- Kết quả chạy:

```
tests > example12.kpl
1 PROGRAM EXAMPLE12;
2 VAR A : INTEGER;
3 B:INTEGER;
4 C:INTEGER;
5 BEGIN
6 A:=10;
7 C:=0;
8 FOR B:=1 to A do
9   if B/2*2 !=0 then C:=C+B;
10 END.
11
```

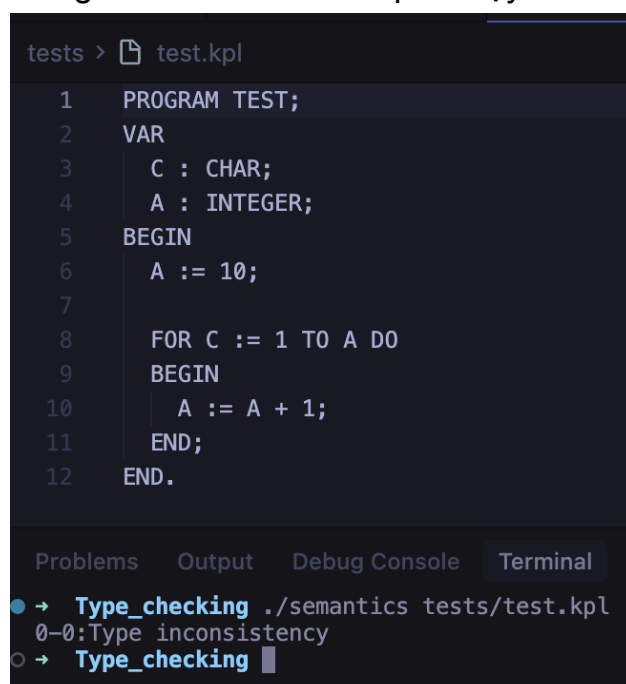
Problems Output Debug Console Terminal Ports Spell Checker 21

• → **Type_checking** ./semantics tests/example12.kpl
 Program EXAMPLE12
 Var A : Int
 Var B : Int
 Var C : Int
 ○ → **Type_checking** █

- Kết quả cho thấy code in ra cấu trúc chương trình và bảng ký hiệu.
- Các biến A, B và C đều được khai báo với kiểu cơ bản integer, do đó việc khai báo là hợp lệ và không phát sinh lỗi kiểu.
- Các lệnh gán $A := 10$ và $C := 0$ đều thỏa mãn quy tắc kiểm tra kiểu vì vế trái và vế phải có cùng kiểu integer.
- Trong lệnh lặp $\text{FOR } B := 1 \text{ TO } A \text{ DO}$, biến điều khiển B và hai biểu thức biên 1 và A đều có kiểu integer, nên lệnh for là hợp lệ về mặt kiểm tra kiểu.
- Ở câu lệnh điều kiện $\text{IF } B/2*2 \neq 0 \text{ THEN}$, biểu thức $B/2*2$ và hằng số 0 đều có kiểu integer, do đó phép so sánh là hợp lệ.
- Cuối cùng, lệnh gán $C := C + B$ trong thân câu lệnh if cũng hợp lệ vì các toán hạng đều có kiểu integer và biểu thức cho kết quả cùng kiểu.

III. Kiểm tra khả năng điều khiển lệnh for bằng biến ký tự

- Mã nguồn file test và kết quả chạy:



The screenshot shows a code editor with a file named `test.kpl` open. The code is as follows:

```

1  PROGRAM TEST;
2  VAR
3      C : CHAR;
4      A : INTEGER;
5  BEGIN
6      A := 10;
7
8      FOR C := 1 TO A DO
9          BEGIN
10             A := A + 1;
11          END;
12  END.

```

Below the code editor, there is a terminal window showing the output of the program. The output is:

```

Type_checking ./semantics tests/test.kpl
0-0:Type inconsistency
Type_checking

```

- Giải thích lỗi semantics: Theo đặc tả của trình biên dịch KPL trong hàm **compileForSt()**:

```

void compileForSt(void) {
    Object* var;
    Type* varType;
    Type* exp1Type;
    Type* exp2Type;
    Token* savedToken;

    eat(KW_FOR);
    eat(TK_IDENT);

    savedToken = currentToken;
    var = checkDeclaredVariable(currentToken->string);
    varType = var->varAttrs->type;
    checkBasicType(varType);

    eat(SB_ASSIGN);
    exp1Type = compileExpression();
    currentToken = savedToken;
    checkTypeEquality(varType, exp1Type);

    eat(KW_TO);
    exp2Type = compileExpression();
    currentToken = savedToken;
    checkTypeEquality(varType, exp2Type);

    eat(KW_DO);
    compileStatement();
}

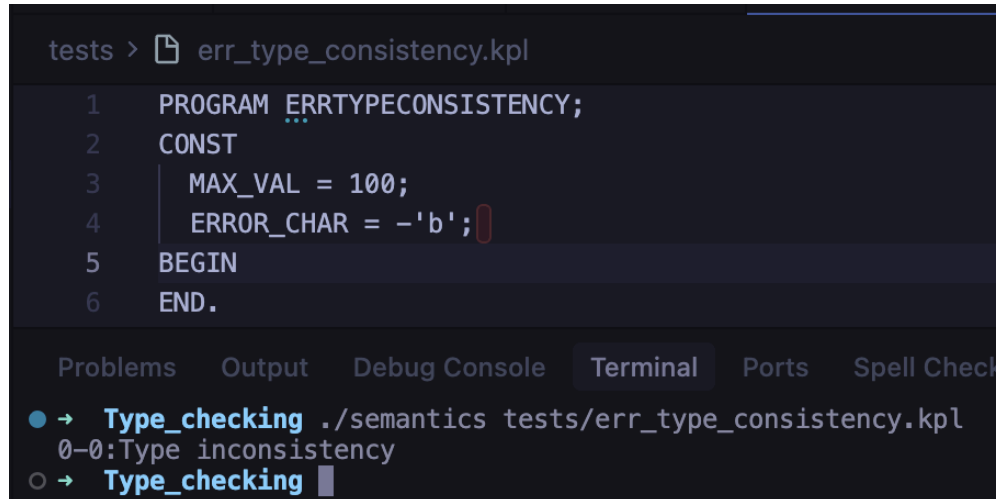
```

- **Bước 1:** Kiểm tra biến điều khiển:
 - Hàm **checkDeclaredVariable()** xác nhận biến C đã được khai báo.
 - **varType** lấy giá trị kiểu của C là **CHAR**.
- **Bước 2:** Kiểm tra biểu thức khởi đầu (exp1Type):
 - Chương trình thực hiện gán C := 1. Biểu thức 1 có kiểu **INTEGER**.
 - Hàm **checkTypeEquality(varType, exp1Type)** thực hiện so sánh kiểu giữa C (**CHAR**) và 1 (**INTEGER**).
 - **Kết quả:** Do 2 kiểu dữ liệu không tương thích, trình biên dịch báo lỗi **Type Inconsistency**.
- Kết quả phân tích:
 - **Vị trí lỗi:** Dòng 7 (lệnh FOR).
 - **Nguyên nhân:** Biến điều khiển C kiểu ký tự không thể nhận giá trị khởi tạo kiểu số nguyên, vi phạm quy tắc đồng nhất kiểu dữ liệu trong phép gán của vòng lặp.
 - **Hành động của Compiler:** Quá trình phân tích ngữ nghĩa bị dừng và thông báo lỗi được in ra.

IV. Thay đổi trong các ví dụ đã cho để gây các lỗi

- **Không tương ứng kiểu khi khai báo hằng:**

- File test và kết quả chạy:



```
tests > err_type_consistency.kpl
1 PROGRAM ERRTYPECONSISTENCY;
2 CONST
3     MAX_VAL = 100;
4     ERROR_CHAR = -'b';
5 BEGIN
6 END.
```

Problems Output Debug Console Terminal Ports Spell Check

- → **Type_checking** ./semantics tests/err_type_consistency.kpl
0-0:Type inconsistency
- → **Type_checking**

- Giải thích: Trong quá trình phân tích ngữ nghĩa (Semantic Analysis) cho hằng số **ERROR_CHAR**, trình biên dịch thực hiện các bước sau:
 - **Phân tích biểu thức:** Nhận diện biểu thức gán hằng là **-'b'**.
 - **Kiểm tra quy tắc toán tử:** Xác định toán tử **-** chỉ hợp lệ với kiểu dữ liệu **INTEGER**.
 - **Đối chiếu kiểu dữ liệu:** Nhận thấy toán hạng **'b'** thuộc kiểu **CHAR**.
 - **Kết luận:** Do có sự xung đột giữa **CHAR** và **INTEGER**, trình biên dịch báo lỗi: **Type inconsistency** (Lỗi không tương thích kiểu).
- Code kiểm tra:



```
case SB_MINUS:
    eat(SB_MINUS);
    constValue = compileConstant2();
    if (constValue->type == TP_INT) {
        constValue->intValue = -constValue->intValue;
    } else {
        error(ERR_TYPE_INCONSISTENCY, currentToken->lineNo, currentToken->colNo);
    }
    break;
```

- Dùng sai số chiều của mảng so với khai báo:

- File test và kết quả chạy:

```
tests >  err_invalid_dimension.kpl

1  PROGRAM ERRINVALIDDIM;
2  VAR
3    LIST : ARRAY(. 20 .) OF CHAR;
4  BEGIN
5    LIST(. 5 .)(. 1 .) := 'A';
6  END.
```

Problems Output Debug Console **Terminal** Ports Spell Checker

- → **Type_checking** ./semantics tests/err_invalid_dimension.kpl
5-17:Type inconsistency
- → **Type_checking** 

- Giải thích: Quá trình kiểm tra lỗi diễn ra qua các bước sau:
 - **Bước 1:** Phân tích chỉ số đầu tiên (. 5 .)
 - Hàm **compileIndexes()** bắt đầu xử lý chỉ số khi gặp token (.).
 - Sau khi kiểm tra chỉ số đầu tiên hợp lệ, kiểu dữ liệu hiện tại được cập nhật từ **ARRAY** sang kiểu phần tử của nó là **CHAR**.
 - **Bước 2:** Phát hiện lỗi tại chỉ số thứ 2 (. 1 .)
 - Parser tiếp tục gặp dấu (. và cố gắng xử lý chỉ số tiếp theo.
 - Lúc này, kiểu hiện tại đã là **CHAR** (không còn là **ARRAY**), dẫn đến điều kiện kiểm tra `arrayType->typeClass != TP_ARRAY` trong hàm **compileIndexes()** bị kích hoạt.
 - Compiler kết luận đây là hành vi truy cập sai số chiều (truy cập chỉ số trên một kiểu dữ liệu đơn).
 - **Bước 3:** Báo lỗi và kết thúc
 - Compiler phát sinh lỗi **ERR_TYPE_INCONSISTENCY** ngay tại vị trí truy cập thừa.
 - Hàm **compileIndexes()** trả về NULL để đánh dấu lỗi ngữ nghĩa và dừng xử lý câu lệnh hiện tại.

- Code kiểm tra:

```
Type* compileIndexes(Type* arrayType) {
    Type* indexType;

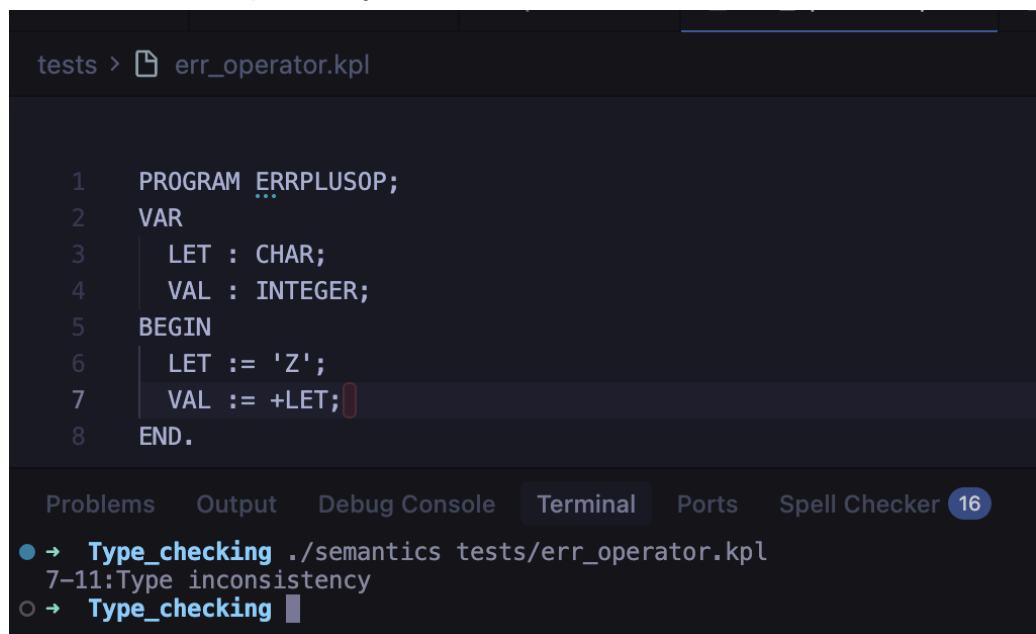
    while (lookAhead->tokenType == SB_LSEL) {
        eat(SB_LSEL);
        indexType = compileExpression();
        checkIntType(indexType);

        if (arrayType == NULL) {
            error(ERR_TYPE_INCONSISTENCY, currentToken->lineNo, currentToken->colNo);
            eat(SB_RSEL);
            return NULL;
        }

        if (arrayType->typeClass != TP_ARRAY) {
            error(ERR_TYPE_INCONSISTENCY, currentToken->lineNo, currentToken->colNo);
            eat(SB_RSEL);
            return NULL;
        }

        arrayType = arrayType->elementType;
        eat(SB_RSEL);
    }
    return arrayType;
}
```

- Một biểu thức bắt đầu bằng + nhưng phần sau đó lại có kiểu ký tự:
 - File test và kết quả chạy:



The screenshot shows an IDE with a file named 'err_operator.kpl' open. The code in the file is as follows:

```
1 PROGRAM ERRPLUSOP;
2 VAR
3   LET : CHAR;
4   VAL : INTEGER;
5 BEGIN
6   LET := 'Z';
7   VAL := +LET;
8 END.
```

The terminal output shows the following error message:

```
● → Type_checking ./semantics tests/err_operator.kpl
7-11:Type inconsistency
○ → Type_checking
```

- Giải thích: Quá trình phân tích ngữ nghĩa cho lỗi sử dụng toán tử + sai kiểu được thực hiện như sau:

- **Phân tích câu lệnh gán:** Khi parser gặp lệnh VAL := +LET, hàm **compileAssignSt()** được kích hoạt để xử lý phép gán.
 - **Xử lý biểu thức về phải:** Sau dấu :=, hàm **compileExpression()** tiến hành phân tích biểu thức +LET.
 - **Xác định kiểu của toán hạng:**
 - Hàm **compileFactor()** nhận diện LET là biến đã khai báo với kiểu **CHAR**.
 - Do đó, hàm **compileExpression2()** trả về kết quả kiểu dữ liệu là **CHAR** cho toán hạng này.
 - Kiểm tra điều kiện toán tử:
 - Trình biên dịch quay lại **compileExpression()** và gọi hàm **checkIntType(type)** để kiểm tra tính hợp lệ của toán tử +.
 - Vì kiểu hiện tại là **CHAR** (không phải **INTEGER**), điều kiện kiểm tra bị vi phạm.
 - **Báo lỗi:** Compiler phát sinh lỗi **ERR_TYPE_INCONSISTENCY**, lập tức dừng xử lý biểu thức và in thông báo lỗi ra màn hình.
- Code kiểm tra:

```
Type* compileExpression(void) {
    Type* type;

    if (lookAhead->tokenType == SB_PLUS) {
        eat(SB_PLUS);
        type = compileExpression2();
        checkIntType(type);
    } else if (lookAhead->tokenType == SB_MINUS) {
        eat(SB_MINUS);
        type = compileExpression2();
        checkIntType(type);
    } else {
        type = compileExpression2();
    }
    return type;
}
```


- **Không tương ứng kiểu trong lệnh gán:**
 - File test và kết quả chạy:

```

tests > err_assign_mismatch.kpl

1 PROGRAM ASSIGNMISMATCH;
2 VAR
3   COUNT : INTEGER;
4 BEGIN
5   COUNT := 'X';
6 END.

Problems Output Debug Console Terminal Ports Spell Checker 15
● → Type_checking ./semantics tests/err_assign_mismatch.kpl
  0-0:Type inconsistency
○ → Type_checking

```

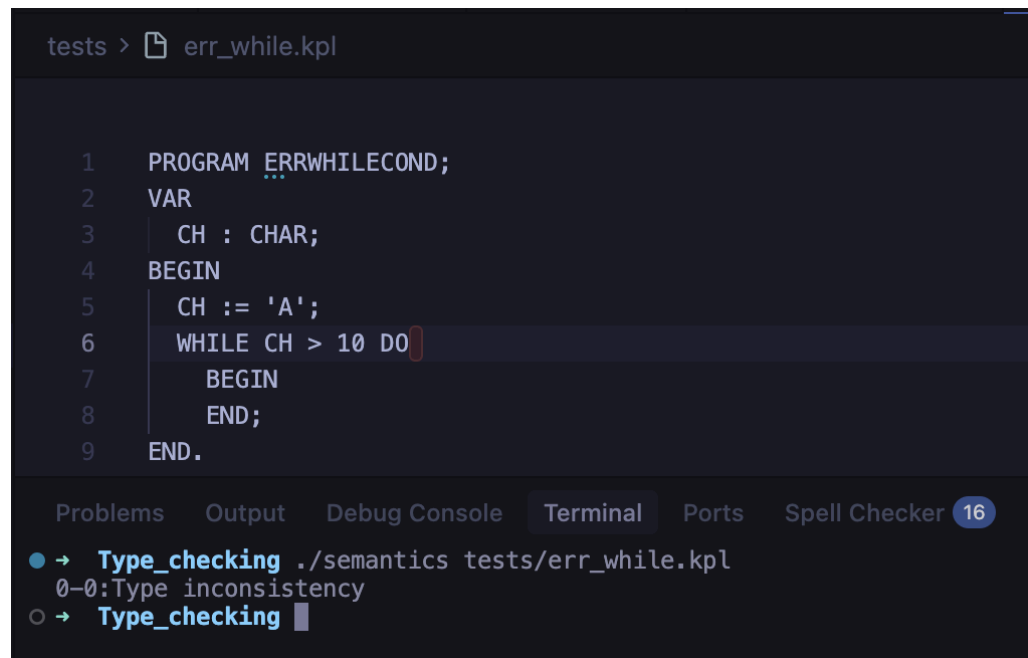
- Giải thích: Quá trình kiểm tra tính hợp lệ của lệnh gán được thực hiện qua các bước sau:
 - **Phân tích về trái (LHS):** Hàm **compileLValue()** xác định biến **COUNT** đã được khai báo với kiểu **INTEGER**. Kiểu này được lưu vào biến **lhsType**.
 - **Phân tích về phải (RHS):** Parser gọi **compileExpression()**. Tại đây, hàm **compileFactor()** nhận diện 'X' là một hằng ký tự và xác định kiểu của vế phải là **CHAR**. Kiểu này được lưu vào biến **rhsType**.
 - Kiểm tra tương thích kiểu:
 - Hàm **compileAssignSt()** gọi hàm hỗ trợ **checkTypeEquality(lhsType, rhsType)** để so sánh hai vế.
 - Vì quy tắc ngôn ngữ KPL không có ép kiểu ngầm định, việc so sánh giữa **INTEGER** và **CHAR** trả về kết quả không khớp (INTEGER khác CHAR).
 - **Báo lỗi:** Compiler phát sinh lỗi **ERR_TYPE_INCONSISTENCY** ngay lập tức dừng xử lý câu lệnh và in thông báo lỗi.

- Code kiểm tra:

```
void compileAssignSt(void) {  
    Type* lhsType;  
    Type* rhsType;  
    Token* savedToken;  
  
    lhsType = compileLValue();  
    savedToken = currentToken;  
    eat(SB_ASSIGN);  
    rhsType = compileExpression();  
    currentToken = savedToken;  
    checkTypeEquality(lhsType, rhsType);  
}
```

- Không tương thích kiểu trong lệnh while:

- File test và kết quả chạy:



```
tests > err_while.kpl  
  
1 PROGRAM ERRWHILECOND;  
2 VAR  
3   CH : CHAR;  
4 BEGIN  
5   CH := 'A';  
6   WHILE CH > 10 DO  
7     BEGIN  
8     END;  
9 END.  
  
Problems Output Debug Console Terminal Ports Spell Checker 16  
● → Type_checking ./semantics tests/err_while.kpl  
0-0:Type inconsistency  
○ → Type_checking
```

- Giải thích:

- **Phân tích lệnh:** Khi gặp lệnh WHILE, trình biên dịch gọi hàm compileWhileSt() và sử dụng compileCondition() để xử lý biểu thức điều kiện.

- **Xác định kiểu toán hạng:**
 - Toán hạng trái: **compileExpression()** xác định biến CH có kiểu CHAR (lưu vào type1).
 - Toán tử: Parser nhận diện toán tử so sánh >.
 - Toán hạng phải: **compileExpression()** xác định giá trị 10 có kiểu INTEGER (lưu vào type2).
 - Kiểm tra tính hợp lệ trong **compileCondition()**:
 - **Kiểu cơ bản:** Hàm **checkBasicType()** xác nhận cả hai đều thuộc nhóm kiểu cơ bản (INTEGER/CHAR).
 - **Sự tương thích:** Hàm **checkTypeEquality(type1, type2)** so sánh kiểu của hai vế. Vì CHAR khác INTEGER => điều kiện này bị vi phạm.
 - **Kết luận:** Compiler phát sinh lỗi **ERR_TYPE_INCONSISTENCY**, dừng xử lý câu lệnh và in thông báo lỗi.
- Code kiểm tra:
- Xử lý điều kiện while:

```
void compileWhileSt(void) {
    eat(KW_WHILE);
    compileCondition();
    eat(KW_DO);
    compileStatement();
}
```

- Kiểm tra điều kiện:

```
void compileCondition(void) {
    Type* type1;
    Type* type2;
    Token* savedToken;

    type1 = compileExpression();
    savedToken = lookAhead;

    switch (lookAhead->tokenType) {
        case SB_EQ:
            eat(SB_EQ);
            break;
        case SB_NEQ:
            eat(SB_NEQ);
            break;
        case SB_LE:
            eat(SB_LE);
            break;
        case SB_LT:
            eat(SB_LT);
            break;
        case SB_GE:
            eat(SB_GE);
            break;
        case SB_GT:
            eat(SB_GT);
            break;
        default:
            error(ERR_INVALID_COMPARATOR, lookAhead->lineNo, lookAhead->colNo);
    }

    type2 = compileExpression();
    currentToken = savedToken;
    checkBasicType(type1);
    checkBasicType(type2);
    checkTypeEquality(type1, type2);
}
```

- Nơi phát sinh lỗi:

```
void checkTypeEquality(Type* type1, Type* type2) {
    if (type1 == NULL || type2 == NULL) {
        error(ERR_TYPE_INCONSISTENCY, currentToken->lineNo, currentToken->colNo);
        return;
    }
    if (compareType(type1, type2) == 0) {
        error(ERR_TYPE_INCONSISTENCY, currentToken->lineNo, currentToken->colNo);
    }
}
```